Security Level
Client Confidential

PENETRATION TESTING REPORT FOR

[redacted]

Issued by	Updated	Approved by
[redacted]	[redacted]	[redacted]

[redacted]

Table of Contents

[redacted]

1. Mission statement

1.1. Introduction

eBuilder Security has been contracted by [redacted] to perform a penetration test on [redacted]

2. Summary of Penetration Test

2.1. Implementation

Evaluation type: Web Application and API security audit
Initial review period: 2025/01/27 to 2025/02/10
WEB : [redacted]
API : [redacted]

2.2. Identified problems

This report details the findings and recommendations arising from the Web Application and API Security Review in determining possible security weaknesses that may exist within their current infrastructure.

There were **12** total numbers of findings identified during the initial review. The following charts represent the breakdown of the risk levels, and the security rating identified after the initial testing.

Issue Status		
Web	API	Infrastructure
8	*4*	-

2.3. Categorization of Observations

The observations are categorized based on their potential impact (harm caused) and probability (probability of detection and exploitation), as well as whether they pertain to web application vulnerabilities or API-related vulnerabilities.

Category based on Impact and probability

- **Critical and high impact issues** - Pose the biggest and most immediate threat and should be prioritized and addressed as soon as possible.

- **Medium impact issues** - Exhibit a varying threat level ranging from low to high, therefore priority should be given based on its CVSS score. Medium-sized issues with high CVSS scores should be addressed like how high issues are handled.

- **Low impact issues** - Can be given the lowest priority because they are very difficult to exploit and therefore pose a very small threat.

Category based on type

- **Web application vulnerabilities** - These vulnerabilities are associated with the web application's functionality, design, or implementation. Examples include SQL injection, cross-site scripting (XSS), improper access control, and insecure file uploads. Such issues can compromise the integrity, availability, or confidentiality of the application and its data.

- **API vulnerabilities** - These vulnerabilities arise from flaws in the Application Programming Interface (API) design or implementation. Common examples include broken authentication, excessive data exposure, improper rate limiting, or insecure communication channels. Exploiting these vulnerabilities can allow attackers to bypass security controls, gain unauthorized access to sensitive information, or disrupt services.

2.4. Compilation of WEB Application observations

Issue ID	Vulnerability	Severity	Status
[redacted]	[redacted]	High	New
[redacted]	[redacted]	High	New
[redacted]	[redacted]	High	New
[redacted]	[redacted]	Medium	New
[redacted]	[redacted]	Medium	New
[redacted]	[redacted]	Medium	New
[redacted]	[redacted]	Low	New
[redacted]	[redacted]	Low	New

2.5. Compilation of API observations

Issue ID	Vulnerability	Severity	Status
[redacted]	[redacted]	High	New
[redacted]	[redacted]	High	New
[redacted]	[redacted]	Medium	New
[redacted]	[redacted]	Medium	New

3. Web-Application Observations

Below is an enumeration of all issues found in the Web Application Security Review. The issues are organized by priority and category and broken down by the package, namespace, or location in which they occur.

The priority of an issue can be Critical, High, Medium, Low or Info.

3.1. [redacted] Web Observations

3.1.1. [redacted]-WEB-STK-2025-01: Lodash Improperly Controlled Modification of Object Prototype Attributes ('Prototype Pollution') Vulnerability.

CVSS Score:	7.5
Severity:	HIGH
Vector String:	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H
Threat:	Prototype Pollution
Root Cause:	Improper Modification Control

3.1.1.1. Observation and impact

A potential Prototype Pollution vulnerability exists due to the use of _.zipObjectDeep in Lodash versions prior to 4.17.20. This function, when used with maliciously crafted input, may allow attackers to modify the Object.prototype, affecting all objects within the application.

Impact includes:

- Potential for arbitrary code execution if the polluted prototype is used in a vulnerable way.
- Denial of service by corrupting core application functionality.
- Circumvention of security mechanisms that rely on the integrity of the object prototype.
- Manipulation of application logic and data flow.

3.1.1.2. Explanation

Prototype pollution is a vulnerability that arises when an attacker can manipulate the prototype of a JavaScript object. In JavaScript, objects inherit properties from their prototypes. By modifying the Object. Prototype, an attacker can inject or modify properties that will be inherited by all subsequently created objects.

3.1.1.3. Identified on

[redacted]

3.1.1.4. Recommendations

1. **Update Lodash:** Upgrade to Lodash version 4.17.20 or later, which includes a patch for this vulnerability. This is the most effective and direct solution.
2. **Input Validation:** Implement strict input validation on all data that is passed to _.zipObjectDeep (or similar deep object creation functions). Sanitize and validate input to prevent malicious keys from being used.
3. **Freeze Prototypes:** Where possible, freeze the Object.prototype using Object.freeze(Object.prototype) to prevent modifications. However, this may break compatibility with some libraries or application code that relies on modifying prototypes.
4. **Object Creation Without Prototypes:** Consider using Object.create(null) for creating objects that do not require prototype inheritance, especially when dealing with user-controlled data.
5. **Code Review:** Perform a thorough code review to identify all instances where _.zipObjectDeep or similar functions are used, and ensure that they are used safely.
6. **Use safer alternatives:** if possible, refactor the code to avoid using lodash functions that are known to have this kind of vulnerability

3.1.2. [redacted]-WEB-2025-02: Lodash Command Injection Vulnerability.

CVSS Score:	7.5
Severity:	HIGH
Vector String:	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H
Threat:	Command Injection
Root Cause:	Inadequate Input Validation

3.1.2.1. Observation and impact

A potential Command Injection vulnerability exists in Lodash versions prior to 4.17.21 due to the template function. This function, when used with user-controlled input, may allow attackers to execute arbitrary commands on the server.

Impact includes:

- Arbitrary code execution on the server.
- Data exfiltration and manipulation.
- System compromise and privilege escalation.
- Denial of service.

3.1.2.2. Explanation

The _.template function in Lodash is used to create compiled templates. In vulnerable versions, the function does not properly sanitize user-provided template strings. If an attacker can control the template string, they can inject malicious code that will be executed when the template is compiled and rendered. This can lead to command injection, where the attacker can execute arbitrary system commands on the server.

3.1.2.3. Identified on

[redacted]

3.1.2.4. Recommendations

1. **Update Lodash:** Upgrade to Lodash version 4.17.21 or later, which includes a patch for this vulnerability. This is the most effective and direct solution.
2. **Avoid User-Controlled Templates:** If possible, avoid using _.template with user-provided input. If templates are necessary, predefine them or use a safer templating engine.
3. **Input Sanitization:** If user-controlled input is unavoidable, implement strict input validation and sanitization. Sanitize all user-provided template strings to remove or escape any potentially dangerous characters or sequences.
4. **Principle of Least Privilege:** Run the application with the least privileges necessary to minimize the impact of a successful command injection attack.
5. **Code Review:** Perform a thorough code review to identify all instances where _.template is used, and ensure that they are used safely. Pay special attention to any cases where user input is used in the template string.
6. **Content Security Policy (CSP):** Implement a strict CSP to limit the execution of dynamically generated code. While it may not fully prevent command injection, it can limit the impact.
7. **Web Application Firewall (WAF):** Deploy a WAF that can detect and block command injection attempts.

3.1.3. [redacted]-WEB-2025-04: Missing HTTP Strict Transport Security (HSTS) Policy.

CVSS Score:	6.5
Severity:	**MEDIUM**
Vector String:	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Threat:	Man-in-the-Middle Attacks
Root Cause:	Missing Security Header

3.1.3.1. Observation and impact

The HTTP Strict Transport Security (HSTS) policy is not enabled. The Strict-Transport-Security header is missing from the server's HTTP responses. This allows attackers to perform Man-in-the-Middle (MitM) attacks by intercepting and modifying HTTP traffic, even if the user initially accessed the site via HTTPS. Impact includes:

- Susceptibility to SSL stripping attacks.
- Potential for attackers to downgrade HTTPS connections to HTTP.
- Increased risk of data interception and manipulation.
- Compromised user trust due to lack of secure connection enforcement.

3.1.3.2. Explanation

HSTS is a security mechanism that forces browsers to interact with a website exclusively over HTTPS. When a browser receives the Strict-Transport-Security header, it remembers that the website should only be accessed via HTTPS for a specified period. Without this header, a user's initial HTTP request could be intercepted and redirected to a malicious site, or an attacker could downgrade the connection to HTTP even if the user initially typed **https://**.

3.1.3.3. Identified on

[redacted]

3.1.3.4. Recommendations

1. **Enable HSTS Header:** Configure the web server to include the Strict-Transport-Security header in all HTTPS responses. Set appropriate values for **max-age**, **includeSubDomains** (if applicable), and preload (if desired).
    - **Example header: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload**
2. **Choose Appropriate max-age:** Start with a shorter max-age value (e.g., a few weeks) and gradually increase it as confidence in the HSTS implementation grows. A value of one year (31536000 seconds) is commonly recommended for production environments.
3. **Consider includeSubDomains:** If all subdomains should also be accessed via HTTPS, include the includeSubDomains directive.
4. **Preload HSTS:** If desired, submit the domain to the HSTS preload list maintained by Google. This will hardcode the HSTS policy into browsers, providing protection even on the first visit.
5. **Test Thoroughly:** After enabling HSTS, thoroughly test the website to ensure that all resources are loaded over HTTPS and that no functionality is broken.
6. **Redirect HTTP to HTTPS:** Ensure that all HTTP requests are redirected to HTTPS. This reinforces the secure connection and prevents users from accidentally accessing the site over HTTP.

3.1.4. [redacted]-WEB-2025- Vulnerable AngularJS Version

CVSS Score:	5.3
Severity:	**MEDIUM**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L
Threat:	Cross-Site Scripting (XSS), Server-Side Request Forgery (SSRF), Other potential vulnerabilities associated with the specific CVEs
Root Cause:	Use of Outdated and Vulnerable Third-Party Library

3.1.4.1. Observation and impact

All versions of the AngularJS package are vulnerable to Cross-Site Scripting (XSS) due to insecure page caching in Internet Explorer. This vulnerability allows the interpolation of <textarea> elements, enabling attackers to inject and execute arbitrary JavaScript code within the context of the affected web page. Impact includes:

- Session hijacking.
- Cookie theft.
- Defacement of the web page.
- Redirection to malicious websites.
- Execution of arbitrary JavaScript code in the user's browser.
- Data theft.

3.1.4.2. Explanation

This application uses AngularJS version 1.8.2, which is known to have multiple vulnerabilities, as evidenced by CVE-2023-26116 and CVE-2022-25869. Using this outdated version exposes the application to potential security risks. Impact includes:

- Potential for attackers to exploit known vulnerabilities for malicious purposes.
- Increased risk of Cross-Site Scripting (XSS) attacks (CVE-2023-26116).
- Possibility of Server-Side Request Forgery (SSRF) attacks (CVE-2022-25869).
- Compromise of user data and application functionality.
- Potential for unauthorized access to internal resources.

3.1.4.3. Identified on

[redacted]

3.1.4.4. Recommendations

1. **Migrate to a Supported Framework:** AngularJS has been deprecated and is no longer maintained. The most effective solution is to migrate to a supported framework like Angular (version 2 or later) or React.
2. **Input Sanitization:** If migration is not immediately feasible, ensure that all user-provided input is properly sanitized and encoded before being used in AngularJS templates, especially within <textarea> elements. Use AngularJS's built-in $sanitize service or a similar library to sanitize HTML.
3. **Content Security Policy (CSP):** Implement a strict CSP to mitigate the impact of XSS attacks. Restrict the sources from which scripts can be loaded and disable unsafe-inline and unsafe-eval.
4. **Avoid Using Interpolation within <textarea>:** If possible, avoid using AngularJS interpolation directly within <textarea> elements. Use alternative methods for displaying and handling user input.
5. **Patch if possible:** Even though the project is deprecated, check if there is any community provided patches, or if the user is using a fork of the project, patch the fork.

3.1.5. [redacted]-WEB-2025-02: Vulnerable jQuery Library (version 3.4.1).

CVSS Score:	**6.1**
Severity:	**MEDIUM**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Threat:	Cross-site Scripting (XSS), Denial of Service(DoS)
Root Cause:	Use of Outdated and Vulnerable Third-Party Library

3.1.5.1. Observation and impact

The application uses jQuery version 3.4.1, which is known to have vulnerabilities. Using outdated JavaScript libraries like this exposes the application to potential security risks. Impact includes:

- Potential for attackers to exploit known vulnerabilities for malicious purposes.
- Increased risk of XSS attacks due to unpatched security flaws.
- Possibility of DoS attacks through specific vulnerabilities.
- Potential for prototype pollution attacks.
- Compromise of user data and application functionality.

3.1.5.2. Explanation

jQuery 3.4.1 has known vulnerabilities that have been addressed in later versions. Vulnerabilities in JavaScript libraries can allow attackers to inject malicious code, manipulate the application's behavior, or cause denial-of-service conditions. Using an outdated version means the application is missing crucial security patches. Specifically, jQuery has had vulnerabilities related to selector parsing, and other areas that could be exploited.

3.1.5.3. Identified on

[redacted]

3.1.5.4. Recommendations

1. **Update jQuery:** Upgrade to the latest stable version of jQuery. This is the most effective way to address known vulnerabilities.
2. **Regularly Update Libraries:** Implement a process for regularly updating all third-party libraries used in the application. Use a dependency management tool like npm or yarn to track and manage library versions.

3.1.6. [redacted]-WEB-2025-03: Vulnerable Lodash Version.

CVSS Score:	5.6
Severity:	**MEDIUM**
Vector String:	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:L
Threat:	Cross-Site Scripting (XSS), Other potential vulnerabilities associated with the specific CVEs
Root Cause:	Use of Outdated and Vulnerable Third-Party Library

3.1.6.1. Observation and impact

The application uses Lodash version 3.10.1, which is known to have multiple vulnerabilities, as evidenced by CVE-2018-3721, CVE-2019-1010266, and CVE-2018-16487. Using this outdated version exposes the application to potential security risks. Impact includes:

- Potential for attackers to exploit known vulnerabilities for malicious purposes.
- Increased risk of Prototype Pollution attacks (CVE-2018-3721).
- Possibility of Regular Expression Denial of Service (ReDoS) attacks (CVE-2019-1010266).
- Potential for Cross-Site Scripting (XSS) attacks (CVE-2018-16487).
- Compromise of user data and application functionality.
- Denial of Service conditions.

3.1.6.2. Explanation

Lodash 3.10.1 contains known vulnerabilities that have been addressed in later versions. These vulnerabilities include prototype pollution, ReDoS, and XSS. Using an outdated version means the application is missing crucial security patches. Specifically:

- **CVE-2018-3721:** Relates to prototype pollution, which can allow attackers to modify the Object.prototype, potentially leading to arbitrary code execution.
- **CVE-2019-1010266:** Relates to ReDoS, which can cause denial-of-service conditions by consuming excessive CPU resources.
- **CVE-2018-16487:** Relates to Cross site scripting, which can allow an attacker to inject malicious code into the client browser.

3.1.6.3. Identified on

[redacted]

3.1.6.4. Recommendations

1. **Update Lodash:** Upgrade to the latest stable version of Lodash. This is the most effective way to address known vulnerabilities.
2. **Regularly Update Libraries:** Implement a process for regularly updating all third-party libraries used in the application. Use a dependency management tool like npm or yarn to track and manage library versions.
3. **Input Validation:** Implement strict input validation and sanitization to prevent potential exploitation of vulnerabilities, especially those related to prototype pollution and ReDoS.
4. **Content Security Policy (CSP):** Implement a strict CSP to mitigate the impact of XSS attacks.

3.1.7. [redacted]-WEB-2025-03: Wildcard Detected in Domain Portion of Content Security Policy (CSP).

CVSS Score:	3.7
Severity:	**Low**
Vector String:	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N
Threat:	Cross-Site Scripting (XSS), Clickjacking Other attacks related to unauthorized framing.
Root Cause:	Misconfiguration (Overly Permissive CSP)

3.1.7.1. Observation and impact

A wildcard was detected in the domain portion of the frame-ancestors CSP directive. This directive controls which domains are allowed to embed the current page in an iframe. The presence of a wildcard indicates that all subdomains of the specified domain are trusted, which may not be the intended behavior. Impact includes:

- Potential for malicious subdomains to embed the page and perform clickjacking attacks.
- Increased risk of XSS attacks if a trusted subdomain is compromised.
- Reduced control over which sites can frame the content, potentially leading to unintended exposure.
- Compromised user trust due to lack of strict control of framing.

3.1.7.2. Explanation

The frame-ancestors CSP directive restricts which domains are allowed to embed the current resource in an iframe, frame, or object. Using a wildcard (e.g., *.example.com) in the domain portion of this directive allows all subdomains of the specified domain to embed the page. While this may be intentional in some cases, it can also create security risks if any of the subdomains are compromised or if the organization does not have full control over all subdomains.

The detail section indicates that the problem was found in the frame-ancestors 'self' [URL] header. The URL contained the wildcard.

3.1.7.3. Identified on

[redacted]

3.1.7.4. Recommendations

1. **Replace Wildcard with Specific Subdomains:** If not all subdomains should be allowed to embed the page, replace the wildcard with the specific subdomains that are trusted.
2. **Review Subdomain Security:** If the wildcard is intentionally used, ensure that all subdomains are properly secured, and that the organization has full control over them. Regularly audit subdomain security.
3. **Use 'self' Directive When Possible:** If the page should only be embedded by the same origin, use the 'self' directive and remove any wildcard entries.
4. **Principle of Least Privilege:** Apply the principle of least privilege by only allowing the necessary domains to embed the page. Avoid using wildcards unless necessary.

3.1.8. [redacted]-WEB-2025-03: Subresource Integrity (SRI) Not Implemented.

CVSS Score:	7.0
Severity:	**MEDIUM**
Vector String:	CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N
Threat:	Man-in-the-Middle (MitM) Attacks, Compromised CDN Attacks, Code Injection
Root Cause:	Misconfiguration (Missing Security Attribute)

3.1.8.1. Observation and impact

Subresource Integrity (SRI) is not implemented for external resources, specifically scripts loaded from the provided URL. This allows attackers to manipulate or replace these resources if they compromise the hosting CDN or perform a Man-in-the-Middle (MitM) attack. Impact includes:

- Potential for malicious code injection into the application.
- Compromised user data and application functionality.
- Increased risk of XSS attacks.
- Potential for attackers to manipulate application behavior.
- Loss of integrity of external assets.

3.1.8.2. Explanation

Subresource Integrity (SRI) is a security mechanism that allows browsers to verify that external resources, such as scripts and stylesheets loaded from CDNs, have not been tampered with. It works by providing a cryptographic hash of the expected resource, which the browser then compares against the downloaded file. Without SRI, attackers can potentially modify these resources to inject malicious code or alter the application's behavior.

The provided URL was detected to be missing the integrity attribute on its <script> tag, which is required for SRI.

3.1.8.3. Identified on

[redacted]

3.1.8.4. Recommendations

1. **Implement SRI:** Add the integrity attribute to all <script> and <link> tags that load external resources. Generate the cryptographic hash of the resource using a strong hash algorithm (e.g., SHA-256, SHA-384, or SHA-512) and include it in the integrity attribute.
    - **Example: <script src="[URL]" integrity="sha384-[HASH]" crossorigin="anonymous"></script>**
2. **Use crossorigin="anonymous":** When using SRI with resources from a different origin, include the crossorigin="anonymous" attribute. This allows the browser to fetch the resource without sending cookies or other user credentials**.**
3. **Generate Hashes:** Use a reliable tool or online service to generate the SRI hashes. Ensure that the hashes are generated from the correct version of the resource.
4. **Regularly Update Hashes:** If the external resource is updated, regenerate the SRI hash and update the integrity attribute accordingly.

3.1.8.5. Supporting evidence

3.1.9. [redacted]-WEB-2025-03: Missing 'object-src' in CSP Declaration.

CVSS Score:	7.0
Severity:	**Low**
Vector String:	CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N
Threat:	Man-in-the-Middle (MitM) Attacks, Compromised CDN Attacks, Code Injection
Root Cause:	Misconfiguration (Missing Security Attribute)

3.1.9.1. Observation and impact

The object-src directive is missing from the Content Security Policy (CSP) declaration. This directive controls the sources from which <object>, <embed>, and <applet> elements can be loaded. Without it, the browser's default behavior is used, which may allow the loading of potentially malicious plugins. Impact includes:

- Potential for attackers to inject malicious plugins into the application.
- Increased risk of XSS attacks through plugin vulnerabilities.
- Possibility of arbitrary code execution through plugins.
- Compromised user data and application functionality.

3.1.9.2. Explanation

The object-src CSP directive restricts the sources from which plugins such as Flash, Silverlight, and Java applets can be loaded. By default, if this directive is not present, the browser may allow the loading of plugins from any source, which can create security risks if a plugin is compromised or if an attacker injects a malicious plugin.

The absence of the directive means that the CSP is not fully protecting the application from plugin-based attacks.

3.1.9.3. Identified on

[redacted]

3.1.9.4. Recommendations

1. **Add object-src Directive:** Explicitly define the object-src directive in the CSP declaration. If no plugins are used, set it to 'none'. If plugins are used, specify the trusted sources.
    - **Example: object-src 'none'; or object-src 'self' https://trusted-plugins.example.com;**
2. **Minimize Plugin Usage:** Evaluate the need for plugins and consider using modern web technologies as alternatives. Plugins are often deprecated and can introduce security risks.

3.1.10. [redacted]-WEB-2025-03: Outdated JavaScript libraries.

CVSS Score:	7.0
Severity:	Low
Vector String:	CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N
Threat:	Cross-Site Scripting (XSS), Denial of Service (DoS)
Root Cause:	Use of Outdated and Vulnerable Third-Party Library

3.1.10.1. Observation and impact

The application is using outdated versions of JavaScript libraries: html5shiv 3.7.0, D3.js 3.5.17, and Modernizr 2.8.3. These outdated versions may contain known vulnerabilities that can be exploited by attackers. Impact includes:

- Potential for attackers to exploit known vulnerabilities for malicious purposes.
- Increased risk of XSS attacks due to unpatched security flaws.
- Possibility of DoS attacks through specific vulnerabilities.
- Potential for prototype pollution attacks.
- Compromise of user data and application functionality.

3.1.10.2. Explanation

Outdated JavaScript libraries like html5shiv, D3.js, and Modernizr may contain known vulnerabilities that have been addressed in later versions. Using these outdated versions means the application is missing crucial security patches. Specific vulnerabilities can vary, but generally, outdated libraries can be exploited for XSS, DoS, and other attacks.

- **html5shiv:** Is used to enable HTML5 elements in older browsers. Although primarily for compatibility, older versions could have issues.
- **D3.js:** Is a powerful data visualization library. Outdated versions may have performance and security issues.
- **Modernizr:** Is a JavaScript library that detects HTML5 and CSS3 features in the user's browser. Older versions may lack security patches and feature detections

.

3.1.10.3. Identified on

[redacted]

3.1.10.4. Recommendations

1. **Update Libraries:** Upgrade html5shiv, D3.js, and Modernizr to their latest stable versions. This is the most effective way to address known vulnerabilities.
2. **Regularly Update Libraries:** Implement a process for regularly updating all third-party libraries used in the application. Use a dependency management tool like npm or yarn to track and manage library versions.

3.1.10.5. Supporting evidence

3.2. [redacted] WEB

3.2.1. [redacted]-WEB-2025-04: Cookie Not Marked as HttpOnly.

CVSS Score:	5.3
Severity:	**MEDIUM**
Vector String:	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Threat:	Cookie Hijacking
Root Cause:	Missing Secure Cookie Flag

3.2.1.1. Observation and impact

Several cookies were identified without the HttpOnly attribute. This exposes them to potential access by client-side scripts, increasing the risk of cookie theft and session hijacking.

3.2.1.2. Explanation

The absence of the HttpOnly flag makes cookies accessible through client-side scripts, potentially enabling attackers to steal session tokens and impersonate legitimate users.

3.2.1.3. Identified on

[redacted]

3.2.1.4. Recommendations

7. **Set HttpOnly Flag:** Configure the web server to mark cookies as HttpOnly to prevent access by client-side scripts.
8. **Secure Flag:** Ensure that cookies are also marked as Secure to enforce transmission only over HTTPS.
9. **Session Management:** Regularly review and monitor cookie configurations to maintain secure session management practices.

4. API Observations

Below is an enumeration of all issues found in API Security Review. The issues are organized by priority and category and broken down by the package, namespace, or location in which they occur.

The priority of an issue can be Critical, High, Medium, Low or Info.

[redacted]

CVSS Score:	
Severity:	**LOW**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Threat:	Stored Cross-Site Scripting (XSS)
Root Cause:	Stored Cross-Site Scripting

4.1.1.1. Observation and impact

The server version "AmazonS3" was found exposed in the HTTP response headers, which can aid attackers in gathering intelligence on the underlying technology. This exposure increases the likelihood of targeted attacks based on known vulnerabilities associated with the disclosed version and assists attackers in refining their strategies, such as fingerprinting the system for further exploitation.

4.1.1.2. Explanation

Server version disclosure occurs when an application includes detailed technology stack information in HTTP response headers. This information can be leveraged by attackers to identify security weaknesses or outdated software versions that have publicly known exploits. Although this does not directly lead to an attack, it contributes to the reconnaissance phase of an attack lifecycle.

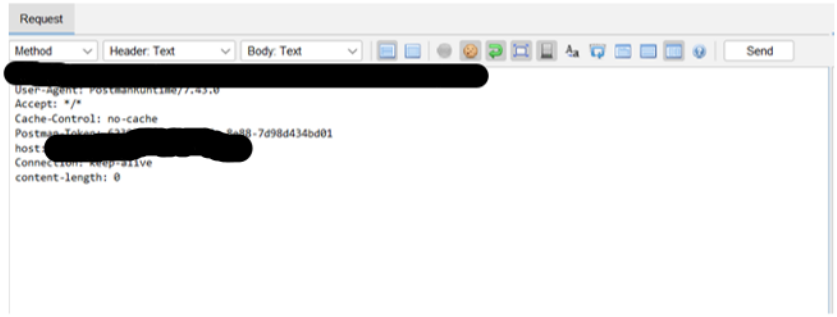
4.1.1.3. Identified on

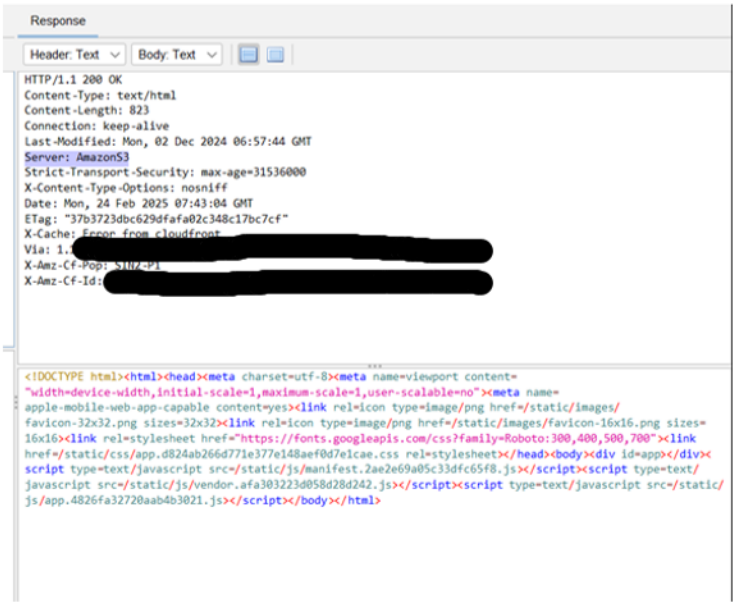
[redacted]

4.1.1.4. Recommendations

- Configure the server to suppress the "Server" header or replace it with a generic value.
- Avoid disclosing detailed version information in response headers, error messages, or metadata.
- Ensure the server software is kept up to date to mitigate risks associated with known vulnerabilities.
- Implement a WAF to inspect and sanitize HTTP responses before they reach the client.

4.1.1.5. Supporting evidence





4.1.2. [redacted]-API-2025-02: Increase the number of requests per client/resource

CVSS Score:	
Severity:	LOW
Vector String:	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:N/A:L
Threat:	Stored Cross-Site Scripting (XSS).
Root Cause:	Stored Cross-Site Scripting

4.1.2.1. Observation and impact

The API allows bulk requests (10,000) per user session without any rate limiting, continuously responding successfully to all requests from the same access token and IP address. This could allow attackers to flood the API with excessive requests, leading to resource exhaustion, and if exploited from multiple machines simultaneously, it could result in a Distributed Denial of Service (DDoS) attack, ultimately degrading system performance and availability for legitimate users.

4.1.2.2. Explanation

Unrestricted resource consumption occurs when an API fails to enforce rate limits, allowing excessive requests from a single user session. Attackers can exploit this weakness to overload system resources, leading to degraded performance or denial of service. If multiple attackers or compromised machines execute such an attack simultaneously, it could result in a large-scale service disruption.

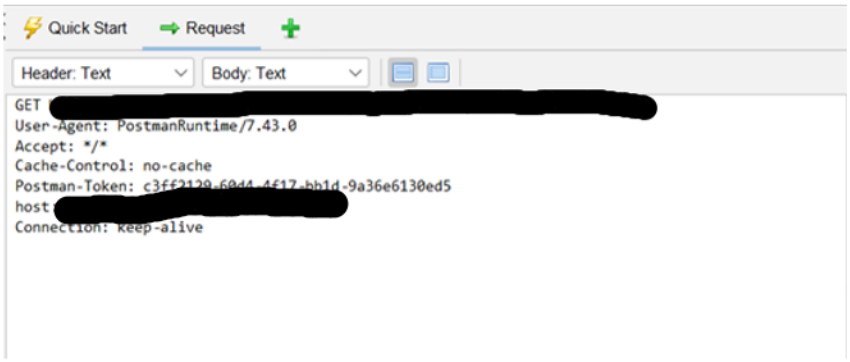
4.1.2.3. Identified on

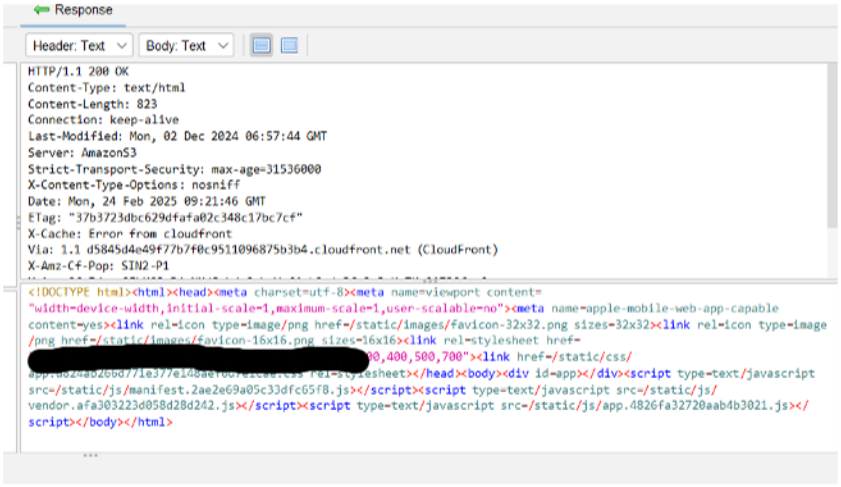
[redacted]

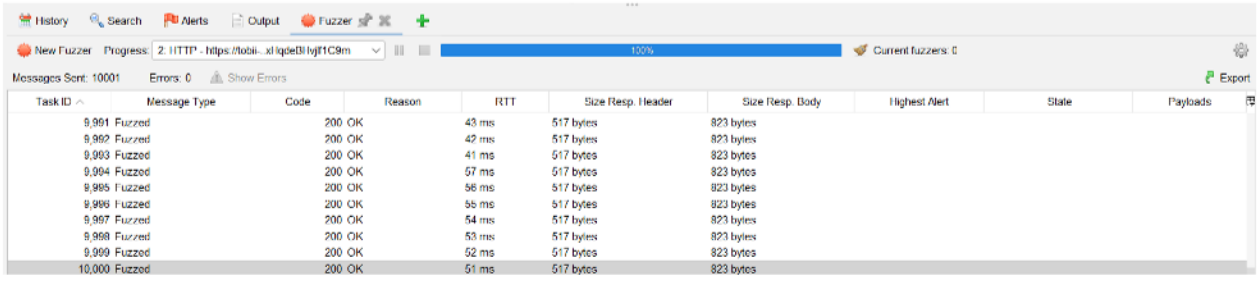
4.1.2.4. Recommendations

- Enforce request limits per user session and IP address using mechanisms like API Gateway rate limiting or web application firewall (WAF) rules.
- Slow down or temporarily block users exceeding a defined threshold to prevent abuse.
- Set up real-time monitoring and alerts for abnormal API request patterns to detect potential abuse early.
- Introduce CAPTCHAs for high-frequency API requests or periodically rotate access tokens to limit prolonged abuse.

4.1.2.5. Supporting evidence







4.1.3. [redacted]-API-2025-03: Cross Origin Resource Sharing : Arbitrary Origin Tursted

CVSS Score:	
Severity:	**LOW**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:N/A:N
Threat:	Denial of Service (DoS) Vulnerability
Root Cause:	Lack of Restrictions on GraphQL Aliases

4.1.3.1. Observation and impact

The Access-Control-Allow-Origin header is set to a wildcard (*), allowing scripts from any origin to access the document tree. This exposes the application to unauthorized cross-origin requests, increases the attack surface for cross-origin attacks, and could potentially allow attackers to access sensitive data via malicious websites or manipulate data.

4.1.3.2. Explanation

Setting the Access-Control-Allow-Origin header to a wildcard allows any external domain to interact with the resources of the application, bypassing security restrictions. This misconfiguration can enable attackers to initiate cross-origin requests, potentially leading to unauthorized access to sensitive information or interactions with the application in unintended ways.

4.1.3.3. Identified on

[redacted]

4.1.3.4. Recommendations

- Specify a limited set of trusted origins for cross-origin requests rather than using a wildcard.
- Implement server-side logic to dynamically check the origin of requests and respond only to those from trusted sources.
- Regularly audit for unauthorized or suspicious cross-origin requests.

4.1.3.5. Supporting evidence

4.1.4. [redacted]-API-2025-04: Missing X-XSS-Protection HTTP Header

CVSS Score:	
Severity:	LOW
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Threat:	Denial of Service (DoS) Vulnerability
Root Cause:	Lack of Restrictions on GraphQL Aliases

4.1.4.1. Observation and impact

The X-XSS-Protection HTTP header is missing in the response, causing the browser to default to behavior that does not prevent rendering, even when a cross-site scripting (XSS) attack is detected. This exposes the application to XSS attacks, potentially leading to data leakage, enabling attackers to execute malicious scripts in the user's browser, and allowing attackers to hijack sessions, steal cookies, or manipulate content.

4.1.4.2. Explanation

The X-XSS-Protection header is designed to instruct the browser to block pages containing reflected XSS attacks. When missing, the browser does not take action to prevent the rendering of malicious content, increasing the risk of XSS attacks that can compromise user data or application functionality.

4.1.4.3. Identified on

[redacted]

4.1.4.4. Recommendations

- Set the X-XSS-Protection header to 1; mode=block to instruct the browser to block detected XSS attacks.
- Implement a strong CSP to further mitigate the risk of XSS attacks by restricting the sources from which scripts can be loaded.
- Perform regular security testing, including XSS vulnerability scans, to identify and fix potential weaknesses.

4.1.4.5. Supporting evidence

Request

Response

[redacted]

5.2.1. [redacted]-API-2025-01: Persistent Authentication Cookie Without Expiration

CVSS Score:	
Severity:	HIGH
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

4.2.1.1. Observation and impact

A persistent authentication cookie was identified during the API security assessment that does not expire. This cookie, generated on February 21st, was reused on February 24th, allowing it to authenticate and feed data into the system without expiration. This issue exposes the application to various security risks, including unauthorized access if the cookie is captured, enabling attackers to perform actions under the legitimate user's identity. Additionally, there is an increased risk of data manipulation, unauthorized data access, or privilege escalation if the attacker holds the valid cookie.

4.2.1.2. Explanation

The vulnerability arises from a lack of expiration for authentication cookies, meaning the session remains active indefinitely unless explicitly terminated. This provides an opportunity for attackers to intercept and reuse authentication cookies to impersonate a user, leading to unauthorized access and potential exploitation of user privileges within the system.

4.2.1.3. Identified on

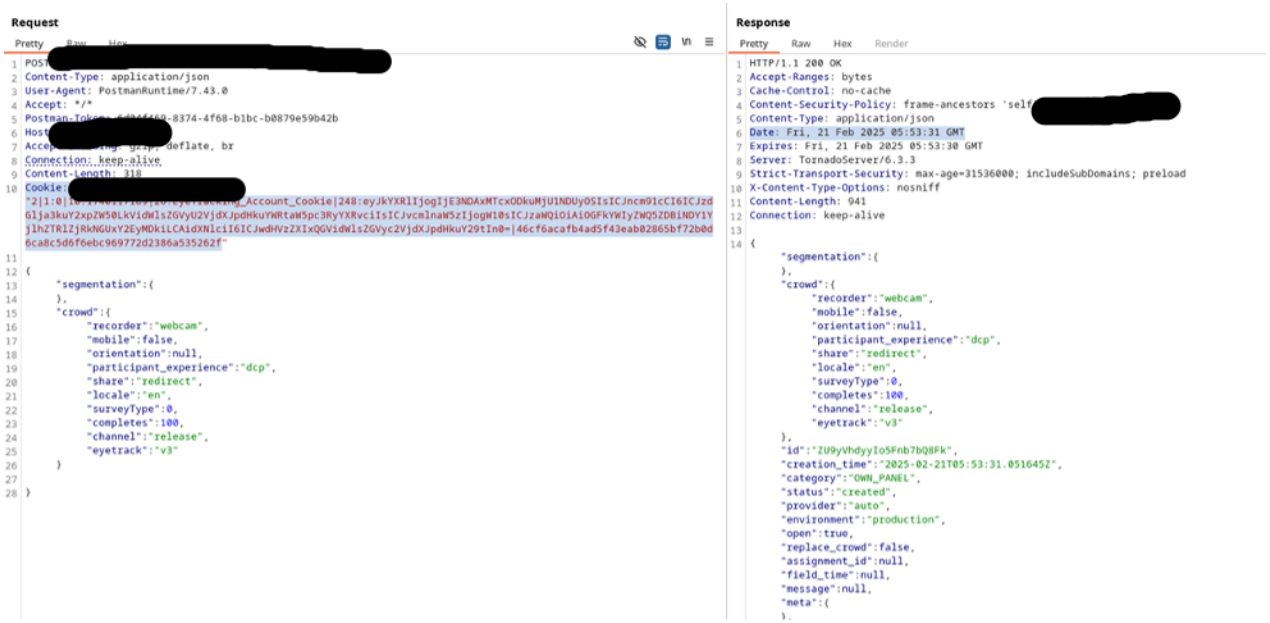
[redacted]

4.2.1.4. Recommendation

- Set a reasonable expiration time for authentication cookies to limit their validity period and reduce the risk of misuse.
- Consider using short-lived tokens (e.g., JWT with expiration) instead of persistent cookies to enhance security and control session validity.
- Maintain session states on the server and allow forced expiration or invalidation of cookies upon logout or after a specified duration.
- Ensure that user logout properly invalidates authentication cookies, preventing reuse."Perform data validation using a single, trustworthy, and actively maintained library.

4.2.1.5. Supporting evidence

Authentication token generated on February 21, 2025.



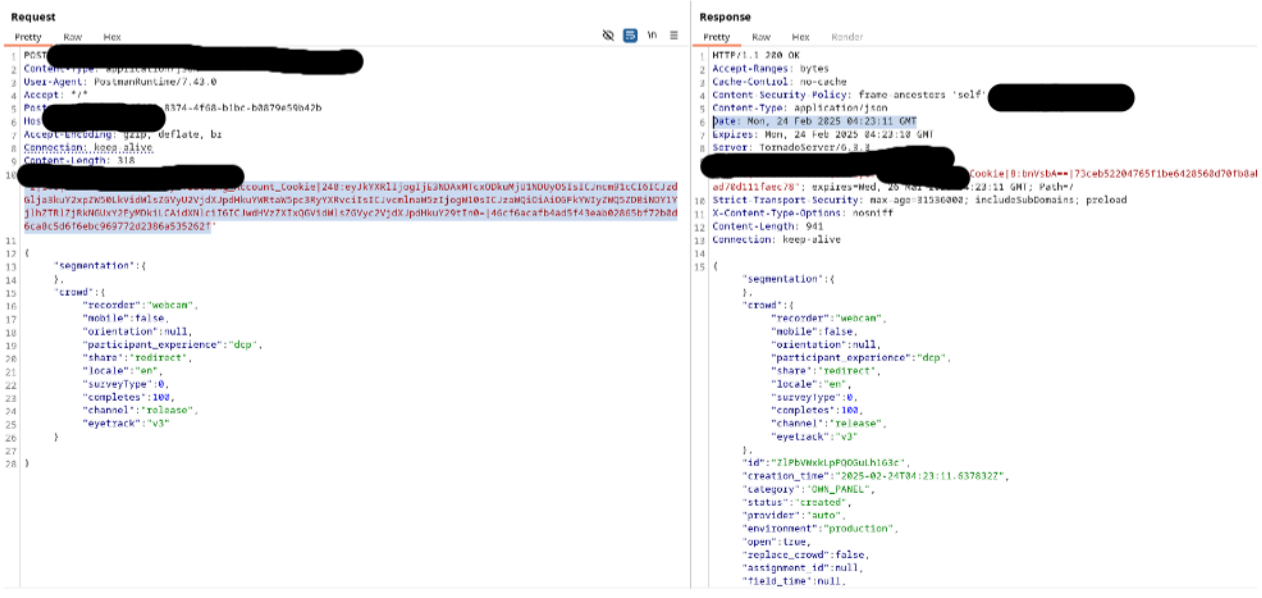
Request

```
POST [redacted]
Content-Type: application/json
User-Agent: PostmanRuntime/7.43.0
Accept: */*
Postman-Token: [redacted]-8374-4f68-b1bc-b0879e59b42b
Host: [redacted]
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 318
Cookie: [redacted]
"2|1:0|[redacted]|248:eyJkYXRlIjogIjE3NDAxMTcxODkuMjU1NDUyOSIsICJncm91cCI6ICJzdGljaskuY2xpZW50LkVidWlsZGVyU2VjdXJpdHkuYWRtaW5pc3RyYXRvciIsICJvcmlnaW5zIjogW10sICJzaWQiOiAiOGFkYWIyZWQ5ZDBiNDY1YjlhZTRlZjRkNGUxY2EyMDkiLCAidXNlciI6ICJwdHVzZXIxQGVidWlsZGVyc2VjdXJpdHkuY29tIn0=|46cf6acafb4ad5f43eab02865bf72b0d6ca8c5d6f6ebc969772d2386a535262f"

{
    "segmentation":{
    },
    "crowd":{
        "recorder":"webcam",
        "mobile":false,
        "orientation":null,
        "participant_experience":"dcp",
        "share":"redirect",
        "locale":"en",
        "surveyType":0,
        "completes":100,
        "channel":"release",
        "eyetrack":"v3"
    }
}
```

Response

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: no-cache
Content-Security-Policy: frame-ancestors 'self' [redacted]
Content-Type: application/json
Date: Fri, 21 Feb 2025 05:53:31 GMT
Expires: Fri, 21 Feb 2025 05:53:30 GMT
Server: TornadoServer/6.3.3
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
Content-Length: 941
Connection: keep-alive

{
    "segmentation":{
    },
    "crowd":{
        "recorder":"webcam",
        "mobile":false,
        "orientation":null,
        "participant_experience":"dcp",
        "share":"redirect",
        "locale":"en",
        "surveyType":0,
        "completes":100,
        "channel":"release",
        "eyetrack":"v3"
    },
    "id":"ZU9yVhdyyIo5Fnb7bQ8Fk",
    "creation_time":"2025-02-21T05:53:31.051645Z",
    "category":"OWN_PANEL",
    "status":"created",
    "provider":"auto",
    "environment":"production",
    "open":true,
    "replace_crowd":false,
    "assignment_id":null,
    "field_time":null,
    "message":null,
    "meta":{
    },
```

Reusing the token generated on February 21, 2025, on February 24, 2025.



Request

```
POST [redacted]
Content-Type: application/json
User-Agent: PostmanRuntime/7.43.0
Accept: */*
Postman-Token: [redacted]-8374-4f68-b1bc-b0879e59b42b
Host: [redacted]
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 318
Cookie: [redacted]
"2|1:0|[redacted]|248:eyJkYXRlIjogIjE3NDAxMTcxODkuMjU1NDUyOSIsICJncm91cCI6ICJzdGljaskuY2xpZW50LkVidWlsZGVyU2VjdXJpdHkuYWRtaW5pc3RyYXRvciIsICJvcmlnaW5zIjogW10sICJzaWQiOiAiOGFkYWIyZWQ5ZDBiNDY1YjlhZTRlZjRkNGUxY2EyMDkiLCAidXNlciI6ICJwdHVzZXIxQGVidWlsZGVyc2VjdXJpdHkuY29tIn0=|46cf6acafb4ad5f43eab02865bf72b0d6ca8c5d6f6ebc969772d2386a535262f"

{
    "segmentation":{
    },
    "crowd":{
        "recorder":"webcam",
        "mobile":false,
        "orientation":null,
        "participant_experience":"dcp",
        "share":"redirect",
        "locale":"en",
        "surveyType":0,
        "completes":100,
        "channel":"release",
        "eyetrack":"v3"
    }
}
```

Response

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: no-cache
Content-Security-Policy: frame-ancestors 'self' [redacted]
Content-Type: application/json
Date: Mon, 24 Feb 2025 04:23:11 GMT
Expires: Mon, 24 Feb 2025 04:23:10 GMT
Server: TornadoServer/6.3.3
[redacted]Cookie|0:bnVsbA==|73ceb52204765f1be6428568d70fb8ad7d11faec78": expires=Wed, 26 Mar 2025 04:23:11 GMT; Path=/
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
Content-Length: 941
Connection: keep-alive

{
    "segmentation":{
    },
    "crowd":{
        "recorder":"webcam",
        "mobile":false,
        "orientation":null,
        "participant_experience":"dcp",
        "share":"redirect",
        "locale":"en",
        "surveyType":0,
        "completes":100,
        "channel":"release",
        "eyetrack":"v3"
    },
    "id":"7lPbVWkKLpFQOGuLh1G3c",
    "creation_time":"2025-02-24T04:23:11.6378322",
    "category":"OWN_PANEL",
    "status":"created",
    "provider":"auto",
    "environment":"production",
    "open":true,
    "replace_crowd":false,
    "assignment_id":null,
    "field_time":null,
```

4.2.2. [redacted] - API:2025-02 - Stored Cross-Site Scripting (XSS)

CVSS Score:	
Severity:	High
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

4.2.2.1. Observation and impact

A Stored Cross-Site Scripting (XSS) vulnerability was found in both file uploads and API responses, where a malicious JavaScript file executed upon file access and the API allowed JavaScript injection. This could lead to attackers executing scripts in the user's browser, hijacking sessions, stealing credentials, performing unauthorized actions, defacing pages, redirecting users to malicious sites, and manipulating or leaking data.

4.2.2.2. Explanation

Stored XSS vulnerabilities occur when user-provided data, such as file names or API input, is improperly sanitized and stored by the server. When this data is later rendered on a web page or reflected in API responses, the malicious script is executed in the context of the victim's session. This enables attackers to execute scripts in the user's browser, which could be used for various malicious activities like stealing cookies, performing unauthorized actions, or injecting phishing content.

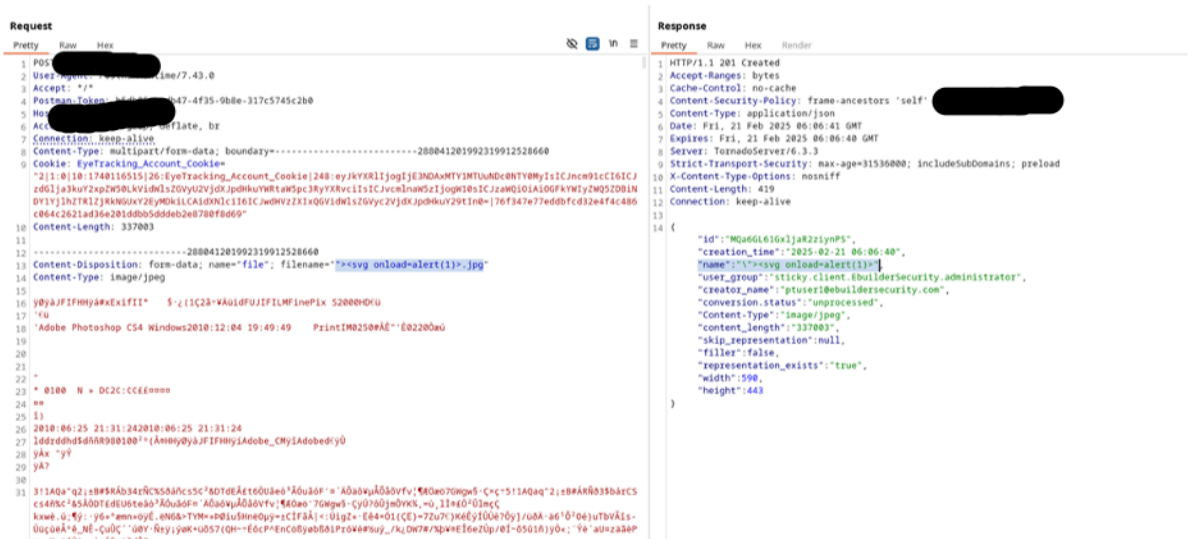
4.2.2.3. Identified on

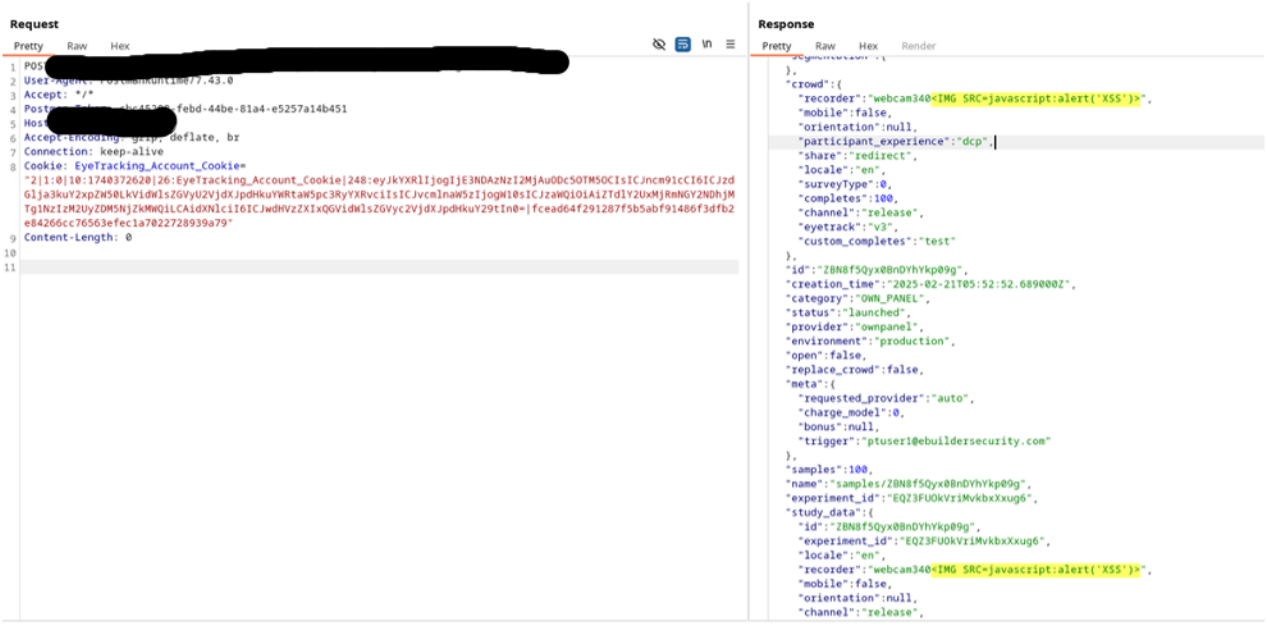
[redacted]

4.2.2.4. Recommendations

- Avoid reflecting the user inputs as it is in the responses without sanitizing.
- Perform data validation using a single, trustworthy, and actively maintained library.
- Validate, filter, and sanitize all client-provided data, or other data coming from integrated systems.
- Special characters should be escaped using the specific syntax for the target interpreter.
- Always limit the number of returned records to prevent mass disclosure in case of injection.
- Define data types and strict patterns for all string parameters
- Use customized error descriptions in responses

4.2.2.5. Supporting evidence





Request

Pretty Raw Hex

```
1 POS[redacted]
2 User-Agent: PostmanRuntime/7.43.0
3 Accept: */*
4 Post[redacted]febd-44be-81a4-e5257a14b451
5 Host[redacted]
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: EyeTracking_Account_Cookie=
"2|1:0|10:1740372620|26:EyeTracking_Account_Cookie|248:eyJkYXRlIjogIjE3NDAzNzI2MjAuODc5OTM5OCIsICJncm91cCI6ICJzdGlja3kuY2xpZW50LkVidWlsZGVyU2VjdXJpdHkuYWRtaW5pc3RyYXRvciIsICJvcmlnaW5zIjogW10sICJzaWQiOiAiZTdlY2UxMjRmNGY2NDhjMTg1NzIzM2UyZDM5NjZkMWQiLCAidXNlciI6ICJwdHVzZXIxQGVidWlsZGVyc2VjdXJpdHkuY29tIn0=|fcead64f291287f5b5abf91486f3dfb2e84266cc76563efec1a7022728939a79"
9 Content-Length: 0
10
11
```

Response

Pretty Raw Hex Render


```
  },
  "crowd":{
    "recorder":"webcam340<IMG SRC=javascript:alert('XSS')>",
    "mobile":false,
    "orientation":null,
    "participant_experience":"dcp",
    "share":"redirect",
    "locale":"en",
    "surveyType":0,
    "completes":100,
    "channel":"release",
    "eyetrack":"v3",
    "custom_completes":"test"
  },
  "id":"Z8N8f5Qyx0BnDYhYkp09g",
  "creation_time":"2025-02-21T05:52:52.689000Z",
  "category":"OWN_PANEL",
  "status":"launched",
  "provider":"ownpanel",
  "environment":"production",
  "open":false,
  "replace_crowd":false,
  "meta":{
    "requested_provider":"auto",
    "charge_model":0,
    "bonus":null,
    "trigger":"ptuser1@ebuildersecurity.com"
  },
  "samples":100,
  "name":"samples/Z8N8f5Qyx0BnDYhYkp09g",
  "experiment_id":"EQZ3FUOkVriMvkbxXxug6",
  "study_data":{
    "id":"Z8N8f5Qyx0BnDYhYkp09g",
    "experiment_id":"EQZ3FUOkVriMvkbxXxug6",
    "locale":"en",
    "recorder":"webcam340<IMG SRC=javascript:alert('XSS')>",
    "mobile":false,
    "orientation":null,
    "channel":"release",
```

5.2.3. [redacted] - API:2025-03 : Reflected Cross-Site Scripting (XSS)

CVSS Score:	
Severity:	Medium
Vector String:	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:L
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

5.2.3.1. Observation and impact

A Reflected XSS vulnerability in the API allowed malicious JavaScript injection through unsanitized input parameters, potentially enabling attackers to steal credentials, perform unauthorized actions, deface pages, redirect users, and manipulate or leak data.

5.2.3.2. Explanation

Reflected XSS vulnerabilities occur when an application reflects untrusted user input, such as query parameters, in the API response without proper sanitization or encoding. If an attacker can inject a malicious script through these inputs, the script is executed in the user's browser when the API response is processed. This allows attackers to execute arbitrary JavaScript within the context of the victim's session.

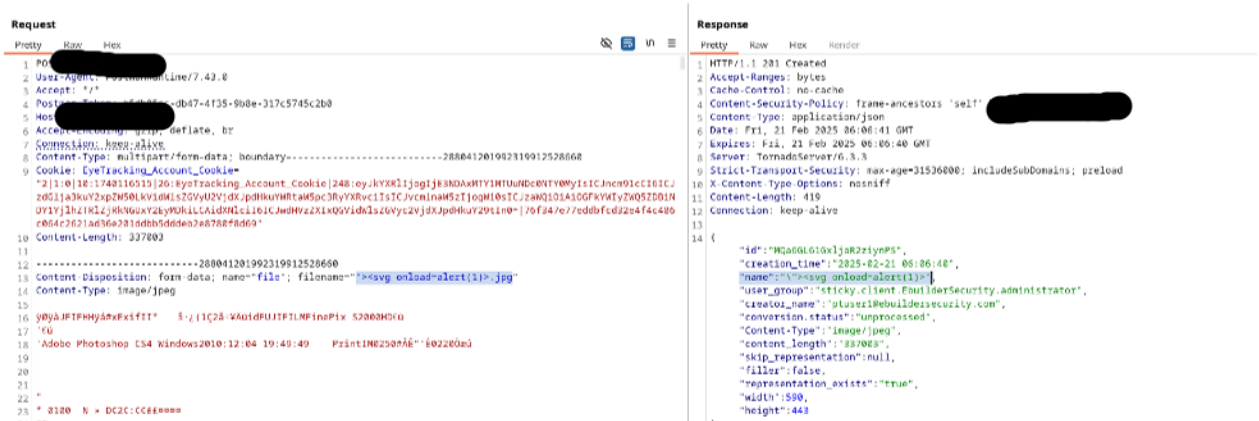
5.2.3.3. Identified on

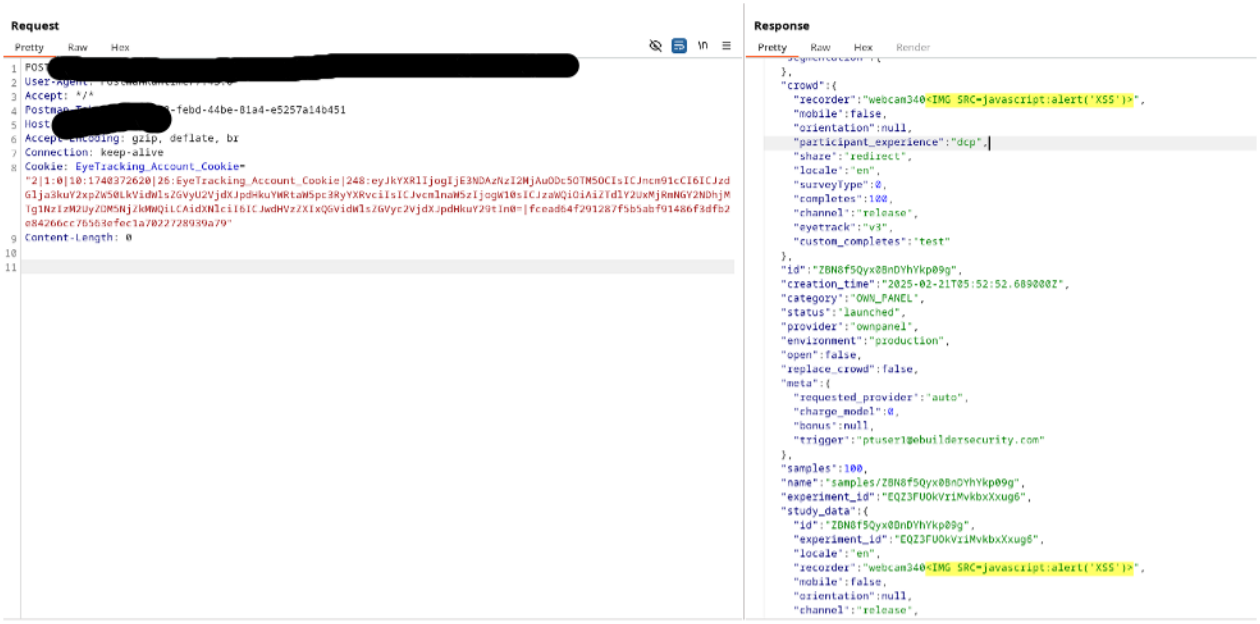
[redacted]

5.2.3.4. Recommendations

- Avoid reflecting the user inputs as it is in the responses without sanitizing.
- Perform data validation using a single, trustworthy, and actively maintained library.
- Validate, filter, and sanitize all client-provided data, or other data coming from integrated systems.
- Special characters should be escaped using the specific syntax for the target interpreter.
- Always limit the number of returned records to prevent mass disclosure in case of injection.
- Define data types and strict patterns for all string parameters
- Use customized error descriptions in responses.

5.2.3.5. Supporting evidence




Request
Pretty Raw Hex
POST [redacted]
User-Agent: [redacted]
Accept: */*
Postman [redacted]-febd-44be-81a4-e5257a14b451
Host [redacted]
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: EyeTracking_Account_Cookie=
"2|1:0|10:1740372620|26:EyeTracking_Account_Cookie|248:eyJkYXRlIjogIjE3NDAzNzI2MjAuODc5OTM5OCIsICJncm91cCI6ICJzdGlja3kuY2xpZW50LkVidWlsZGVyU2VjdXJpdHkuYWRtaW5pc3RyYXRvciIsICJvcmlnaW5zIjogW10sICJzaWQiOiAiZTdlY2UxMjRmNGY2NDhjM2Y1NzIzM2UyZDM5NjZkMWQiLCAidXNlciI6ICJwdHVzZXIxQGVidWlsZGVyc2VjdXJpdHkuY29tIn0=|fcead64f291287f5b5abf91486f3dfb2e84266cc76568efec1a7022728939a79"
Content-Length: 0
Response
Pretty Raw Hex Render
},
"crowd":{
"recorder":"webcam340<IMG SRC=javascript:alert('XSS')>",
"mobile":false,
"orientation":null,
"participant_experience":"dcp",
"share":"redirect",
"locale":"en",
"surveyType":0,
"completes":100,
"channel":"release",
"eyetrack":"v3",
"custom_completes":"test"
},
"id":"ZBN8f5Qyx0BnDYhYkp09g",
"creation_time":"2025-02-21T05:52:52.689000Z",
"category":"OWN_PANEL",
"status":"launched",
"provider":"ownpanel",
"environment":"production",
"open":false,
"replace_crowd":false,
"meta":{
"requested_provider":"auto",
"charge_model":0,
"bonus":null,
"trigger":"ptuser1@ebuildersecurity.com"
},
"samples":100,
"name":"samples/ZBN8f5Qyx0BnDYhYkp09g",
"experiment_id":"EQZ3FUOkViMvkbxXxug6",
"study_data":{
"id":"ZBN8f5Qyx0BnDYhYkp09g",
"experiment_id":"EQZ3FUOkViMvkbxXxug6",
"locale":"en",
"recorder":"webcam340<IMG SRC=javascript:alert('XSS')>",
"mobile":false,
"orientation":null,
"channel":"release",

5.2.4. [redacted]-API-2025-04: Server Version Disclosure in HTTP Headers

CVSS Score:	
Severity:	**LOW**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

5.2.4.1. Observation and impact

The exposure of the TornadoServer/6.3.3 version in the HTTP response headers increases the risk of targeted attacks. Attackers can exploit known vulnerabilities associated with the server version, potentially compromising the system and using the information for further attacks.

5.2.4.2. Explanation

Server version disclosure occurs when the web server exposes details about the software and its version in HTTP response headers. This information can be useful for attackers, as it allows them to look up known vulnerabilities or weaknesses specific to that version, increasing the chances of a successful attack. It is a best practice to minimize the exposure of such details to reduce the attack surface.

5.2.4.3. Identified on

[redacted]

5.2.4.4. Recommendations

- Configure the web server to prevent disclosing version details in HTTP response headers.
- Update TornadoServer to latest satble release (6.4.2)

5.2.4.5. Supporting evidence

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Accept-Ranges: bytes
3  Cache-Control: no-cache
4  Content-Security-Policy: frame-ancestors 'self' [redacted]
5  Content-Type: application/json
6  Date: Mon, 24 Feb 2025 06:21:12 GMT
7  Expires: Mon, 24 Feb 2025 06:21:11 GMT
8  Server: TornadoServer/6.3.3
9  Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
10 X-Content-Type-Options: nosniff
11 Content-Length: 1002
12 Connection: keep-alive
```

5.2.5. [redacted]-API-2025-05: Increase the number of requests per client/resource

CVSS Score:	
Severity:	**LOW**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:N/A:L
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

5.2.5.1. Observation and impact

The API allows excessive requests (10,000 per session) without rate limiting, enabling attackers to exhaust system resources, degrade service, and potentially cause a DDoS-like impact if executed from multiple machines.

5.2.5.2. Explanation

The vulnerability arises from the lack of rate limiting or resource consumption controls, which would normally limit the number of requests a client can make within a certain timeframe. Without these protections, attackers can flood the system with a high volume of requests, consuming resources and potentially causing a denial of service. Since the requests are all validated under the same user session, this issue does not rely on the user's behavior or actions but rather the API's failure to limit resource consumption.

5.2.5.3. Identified on

[redacted]

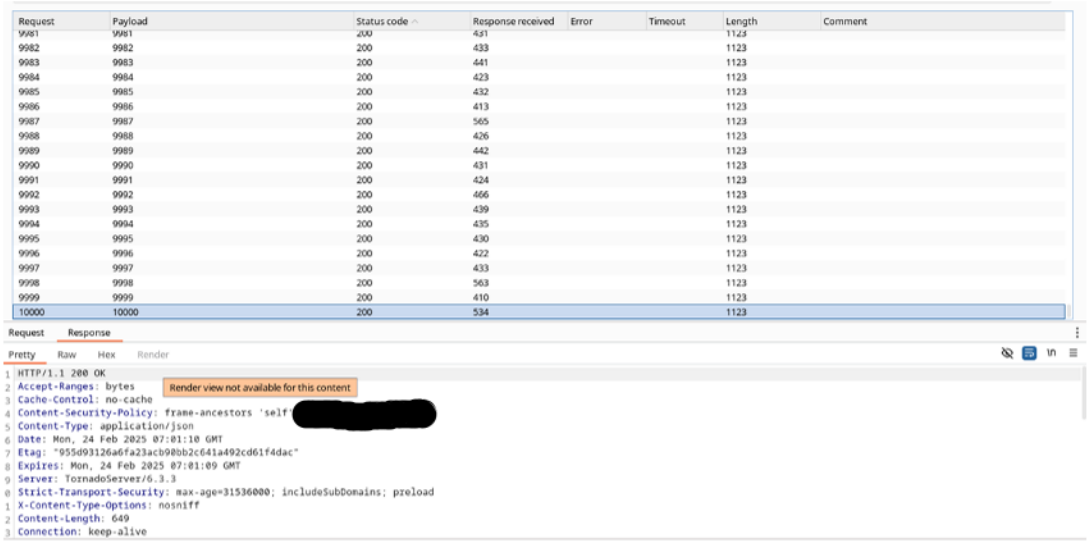
5.2.5.4. Recommendations

Implement a limit on how often a client can call the API within a defined time frame.

- Notify the client when the limit is exceeded by providing the limit number and the time at which the limit will be reset.

- Detect high amounts of requests from the same IP and blacklist IP for a period of time
- Detect high amounts of requests from the same access token in a short period of time and invalidate/expire the access token.

5.2.5.5. Supporting evidence



Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
9981	9981	200	431			1123	
9982	9982	200	433			1123	
9983	9983	200	441			1123	
9984	9984	200	423			1123	
9985	9985	200	432			1123	
9986	9986	200	413			1123	
9987	9987	200	565			1123	
9988	9988	200	426			1123	
9989	9989	200	442			1123	
9990	9990	200	431			1123	
9991	9991	200	424			1123	
9992	9992	200	466			1123	
9993	9993	200	439			1123	
9994	9994	200	435			1123	
9995	9995	200	430			1123	
9996	9996	200	422			1123	
9997	9997	200	433			1123	
9998	9998	200	563			1123	
9999	9999	200	410			1123	
10000	10000	200	534			1123	

Request Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Accept-Ranges: bytes
3 Cache-Control: no-cache
4 Content-Security-Policy: frame-ancestors 'self' [redacted]
5 Content-Type: application/json
6 Date: Mon, 24 Feb 2025 07:01:10 GMT
7 Etag: "955d93126a6fa23acb98bb2c641a492cd61f4dac"
8 Expires: Mon, 24 Feb 2025 07:01:09 GMT
9 Server: TornadoServer/6.3.3
0 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
1 X-Content-Type-Options: nosniff
2 Content-Length: 649
3 Connection: keep-alive
```

Render view not available for this content

5.2.6 [redacted]-API-2025-04: Stored XSS

CVSS Score:	
Severity:	HIGH
Vector String:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

Observation and impact

identified a vulnerability that can send malformed injection code to backend using unsensitized parameters and reflect it through the subsequent response. Certain payloads were stored at the server and reflected in subsequent requests regarding the business flow.

Explanation

XSS vulnerability stems from improper input validation, where the application fails to sanitize or encode user input properly before rendering it on the web page. This enables attackers to inject malicious scripts that execute in the context of the targeted domain.

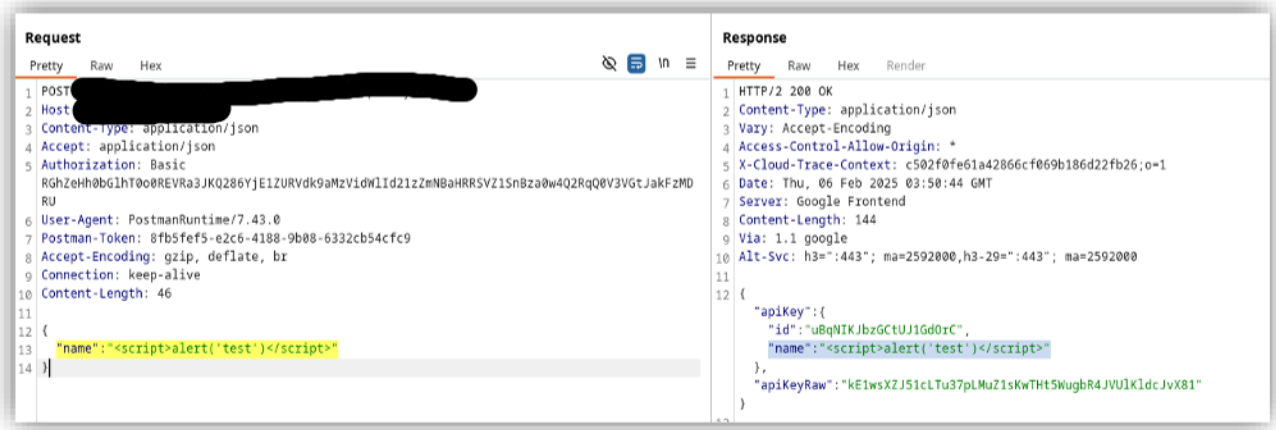
Identified on

[redacted]

Recommendations

- Avoid reflecting the user inputs as it is in the responses without sanitizing.
- Perform data validation using a single, trustworthy, and actively maintained library.
- Validate, filter, and sanitize all client-provided data, or other data coming from integrated systems.
- Special characters should be escaped using the specific syntax for the target interpreter.
- Always limit the number of returned records to prevent mass disclosure in case of injection.
- Define data types and strict patterns for all string parameters
- Use customized error descriptions in responses.

Supporting evidence



5.2.7 [redacted]-API-2025-04: Reflected XSS

CVSS Score:	
Severity:	**MEDIUM**
Vector String:	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:L
Threat:	Cross-Site Scripting (XSS).
Root Cause:	Cross-Site Scripting

Observation and impact

identified a vulnerability that can send malformed injection code to backend using unsensitized parameters and reflect it through the subsequent response. Certain payloads were stored at the server and reflected in subsequent requests regarding the business flow.

Explanation

XSS vulnerability stems from improper input validation, where the application fails to sanitize or encode user input properly before rendering it on the web page. This enables attackers to inject malicious scripts that execute in the context of the targeted domain.

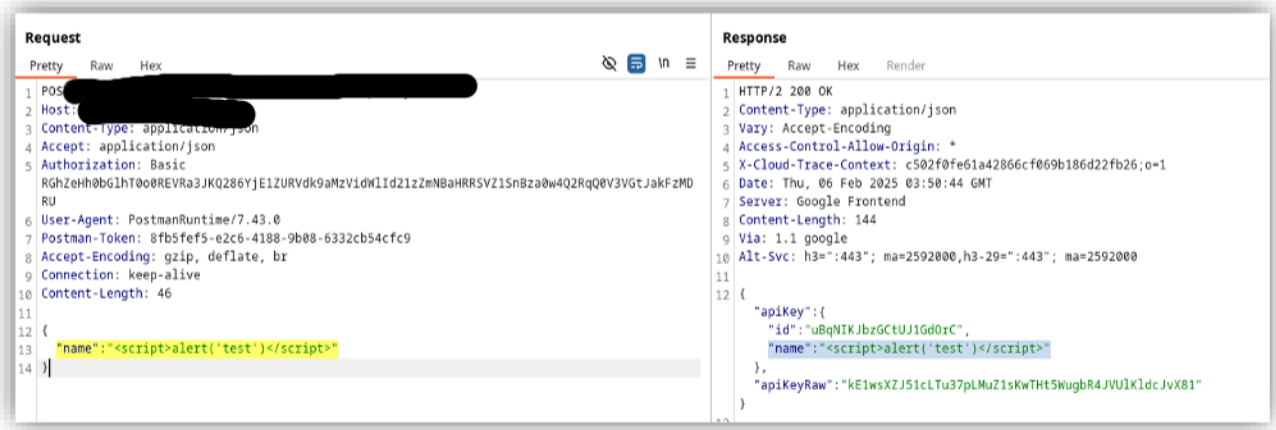
Identified on

- [redacted]
- [redacted]
- [redacted]
- [redacted]
- [redacted]

Recommendations

- Avoid reflecting the user inputs as it is in the responses without sanitizing.
- Perform data validation using a single, trustworthy, and actively maintained library.
- Validate, filter, and sanitize all client-provided data, or other data coming from integrated systems.
- Special characters should be escaped using the specific syntax for the target interpreter.
- Always limit the number of returned records to prevent mass disclosure in case of injection.
- Define data types and strict patterns for all string parameters
- Use customized error descriptions in responses.

Supporting evidence



3. Assumptions and limitations

[redacted]